



5강. Python 프로그래밍 — venv·Jupyter·자료구조·함수·객체 지향·타입 힌트



학습 목표

- venv 가상환경을 만들고 활성화하여 프로젝트별 패키지를 격리 관리할 수 있습니다.
- Jupyter Notebook의 구조와 셀·커널 실행 모델을 이해하고, 실습 환경으로 사용할 수 있습니다.
- list/dict/set/tuple의 특성과 시간 복잡도를 이해하고, 상황에 맞는 자료구조를 선택할 수 있습니다.
- 함수·모듈·패키지로 코드를 구조화하고, 가변 기본 인자 같은 혼한 함정을 피할 수 있습니다.
- 클래스·상속·다형성·매직 메서드로 객체지향 설계를 구현할 수 있습니다.
- 타입 힌트를 적용해 협업과 도구 지원(자동완성·정적 검사)에 유리한 코드를 쓸 수 있습니다.



목차

들어가며

1. venv — 프로젝트마다 독립된 패키지 공간

2. Jupyter Notebook — 실습을 기록으로 남기는 도구

셀은 독립적이지 않습니다

3. 자료구조 — list, tuple, dict, set을 언제 쓰나

시간 복잡도 — 왜 dict/set이 빠를까요?

4. 함수·모듈·패키지 — 코드를 구조화하기

5. 객체지향 — 클래스·상속·다형성·매직 메서드

매직 메서드 — 내 클래스를 파이썬답게

캡슐화 — 공개할 것과 감출 것

6. 타입 힌트 — 협업을 위한 계약서

미니 퀴즈

실습

정리

들어가며

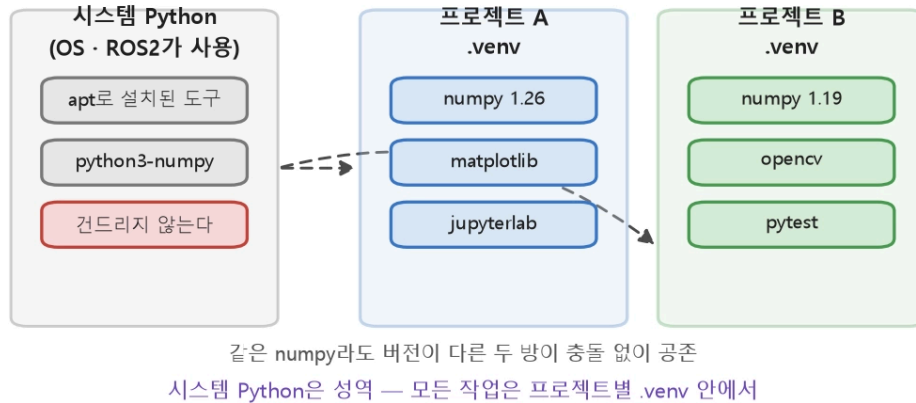
ROS2 노드, 인지 알고리즘, 데이터 분석, 시뮬레이션 — 로봇 개발에서 Python은 거의 모든 곳에 등장합니다. C++보다 느리지만(7장에서 비교합니다) **작성 속도가 빠르고 생태계가 방대**해서, "일단 Python으로 빠르게 만들고, 성능이 필요한 부분만 C++로" 가는 것이 로봇 소프트웨어의 일반적인 전략입니다.

이번 강의는 문법 나열이 아니라, **로봇 코드를 구조적으로 쓰기 위한 Python의 뼈대** — 환경 격리, 자료구조 선택, 코드 구조화, 객체지향, 타입 힌트 — 를 다룹니다. 이 뼈대는 모듈 ②의 ROS2 노드 작성과 모듈 ④의 시뮬레이션 구현에서 그대로 쓰입니다.

1. venv — 프로젝트마다 독립된 패키지 공간

Python 프로젝트를 여러 개 진행하다 보면 반드시 겪는 문제가 있습니다. 프로젝트 A는 `numpy 1.26` 이 필요한데 프로젝트 B는 옛날 코드라 `numpy 1.19` 에서만 돛니다. 시스템에 numpy는 하나만 설치할 수 있으니 **버전 충돌**이 납니다. 더 위험한 것은, 시스템 Python(OS와 ROS2가 쓰는)에 `pip install` 을 남발하다가 **시스템 도구 자체가 망가지는** 경우입니다.

해결책이 **가상환경(venv)** — 프로젝트별로 독립된 Python 패키지 공간을 만드는 표준 도구입니다.



images/05_venv_격리.png — 가상환경 격리

💡 **해석:** 시스템 Python은 건드리지 않고, 프로젝트마다 자기만의 패키지 방 (venv)을 만듭니다. 각 방에는 서로 다른 버전의 패키지가 공존할 수 있고, 방 하나가 꼬여도 지우고 다시 만들면 그만입니다. **"시스템 Python은 성역, 작업은 항상 venv 안에서"**가 원칙입니다.

```
# 생성 (프로젝트 디렉터리에서, 1회) python3 -m venv .venv # 활성화 — 프롬프트에 (.venv)가 붙습니다 source .venv/bin/activate # 리눅스/macOS #  
.venv\Scripts\activate # Windows pip install numpy matplotlib # 이  
제 이 방에만 설치됩니다 pip freeze > requirements.txt # 설치 목록을 기록  
(팀 공유용) deactivate # 방에서 나오기
```

팀원은 `pip install -r requirements.txt` 한 줄로 같은 환경을 재현합니다. **"내 컴퓨터에선 되는데요"**의 상당수가 이 재현 절차 하나로 예방됩니다. 이 가상환경은 모듈 ④의 미니 프로젝트까지 그대로 이어집니다.

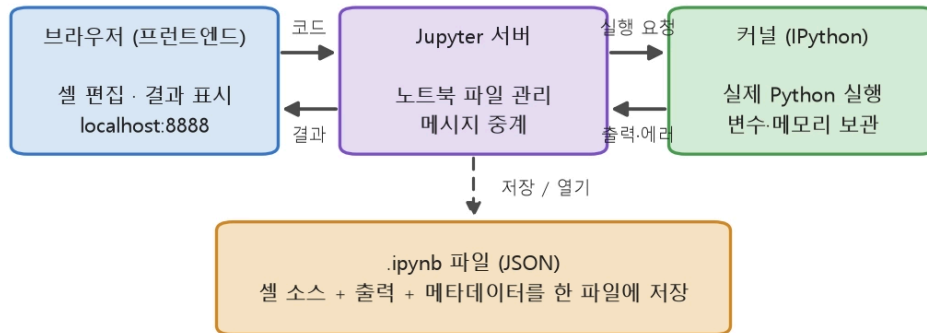
2. Jupyter Notebook — 실습을 기록으로 남기는 도구

로봇 데이터를 다루다 보면 "코드를 짜서 한 번 돌리는" 일보다 **"계산하고, 그려 보고, 값을 바꿔 다시 그려 보는"** 탐색이 훨씬 많습니다. 좌표 변환이 맞는지, 센서 값이 정상인지는 숫자만 봐서는 알 수 없고 그림을 그려 봐야 합니다.

Jupyter Notebook은 이 탐색에 맞춰 만들어진 도구입니다. 코드를 셀 단위로 쪼개 실행하고, 그 결과(숫자·표·그림)가 코드 바로 아래에 남습니다. 6강 NumPy부터 모듈 ③의 실습, 모듈 ④의 미니 프로젝트까지 이 환경에서 진행합니다.

```
# 위에서 만든 .venv 를 활성화한 상태에서 pip install jupyterlab
ipykernel # 이 가상환경을 노트북 커널로 등록 (한 번만) python -m ipykernel
install --user --name lv1 --display-name "Python (lv1)" jupyter
lab # 브라우저가 열립니다
```

Jupyter Notebook의 3층 구조 — 브라우저 · 서버 · 커널



코드는 브라우저에서 쓰지만 실행은 커널이 한다 — 커널이 죽으면 변수도 사라진다

images/05_주피터_구조.png — Jupyter Notebook의 3층 구조

💡 **해석:** 우리가 보는 화면은 **브라우저**일 뿐이고, 코드는 **Jupyter 서버**를 거쳐 **커널**에서 실행됩니다. 변수와 import는 전부 커널의 메모리에 있습니다. 노트북 파일(**.ipynb**)은 셀의 소스와 출력을 함께 담은 JSON 텍스트 파일이라, Git으로 버전 관리도 됩니다(4강).

✅ **중요:** `ipykernel install` 단계를 빠뜨리면 노트북이 **시스템 파이썬**으로 돌아, 분명히 설치했는데 `ImportError` 가 납니다. 셀에서 `import sys; print(sys.executable)` 로 어느 파이썬이 도는지 항상 확인할 수 있습니다.

셀은 독립적이지 않습니다

노트북을 처음 쓸 때 반드시 겪는 함정이 있습니다.

셀 실행 순서 — 노트북에서 가장 흔한 함정



images/05_커널_실행순서.png — 셀 실행 순서

💡 **해석:** 셀은 각각 독립된 프로그램이 아니라 **같은 커널 메모리를 이어서 건드리**는 조각입니다. 위아래로 오가며 실행하면 화면에 보이는 코드 순서와 실제 실행 이력이 어긋나고, 결과를 믿을 수 없게 됩니다. 셀 왼쪽의 실행 카운터 **[n]**이 위에서 아래로 증가하는지가 그 판별 기준입니다.

그래서 제출·공유 전에는 **Kernel** → **Restart Kernel and Run All Cells** 로 위에서 아래까지 한 번에 통과하는지 확인합니다. 이 한 번의 습관이 "내 화면에서는 되던데요"를 예방합니다.

자주 쓰는 단축키는 다섯 개면 충분합니다.

단축키	동작
Shift + Enter	셀 실행 후 다음 셀로 이동
Ctrl + Enter	셀 실행 후 그 자리에 머무름
Esc → A / B	위 / 아래에 새 셀 삽입
Esc → M / Y	마크다운 셀 / 코드 셀로 전환
Esc → D D	셀 삭제

💡 **팁:** 노트북은 **탐색과 설명**에 강하고, 재사용할 코드를 담기에는 약합니다. 두 번 쓰는 코드는 **.py** 모듈로 옮기는 것이 원칙입니다(3절). 모듈 ④에서 이 원칙을 프로젝트 구조로 다룹니다.

3. 자료구조 — list, tuple, dict, set을 언제 쓰나

Python의 4대 내장 자료구조는 각각 **다른 문제**를 푼다. "다 list로" 짜는 코드와 알맞게 고른 코드는 가독성과 성능이 크게 다릅니다.



Go online to view this image

images/05_자료구조_선택.png — 자료구조 선택

💡 **해석:** 순서 있는 목록이면 **list**, 바뀌면 안 되는 값 묶음이면 **tuple**, 이름으로 찾는 표라면 **dict**, 중복 없는 모음·소속 검사라면 **set**입니다. 특히 dict와 set의 조회가 **O(1)** — 데이터가 백만 개여도 한 번에 찾는다는 점이 핵심입니다.

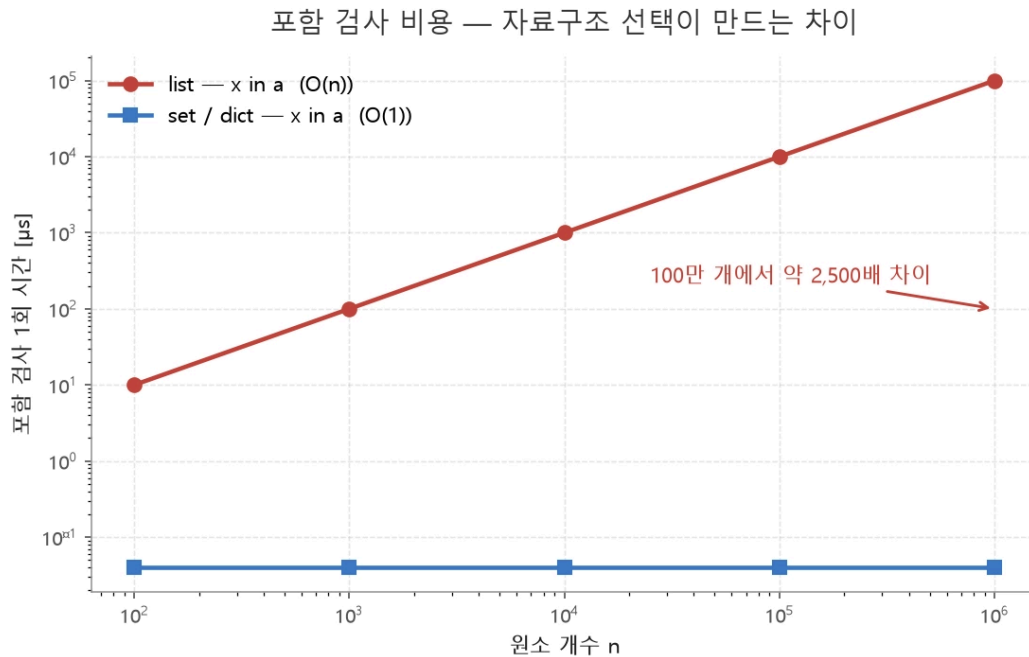
로봇 코드 맥락의 예시로 정리합니다.

```
# list - 순서 있는 시계열, 추가/삭제가 잦다 distance_log = [] # LiDAR 거리 측정 누적 distance_log.append(0.83) # tuple - 불변(immutable) 묶음. 좌표·설정처럼 "바뀌면 안 되는" 값 home_position = (0.0, 0.0, 0.0) # x, y, z # home_position[0] = 1.0 → TypeError! 실수로 바꾸는 것을 언어가 막아줌 # dict - 키로 즉시 조회. 센서 이름 → 최근 값 sensor_values = {"lidar": 0.83, "imu_pitch": 2.1, "battery": 11.9} sensor_values["battery"] # O(1) 조회 # set - 중복 없는 모음, 포함 검사가 빠름 detected_ids = {3, 7, 12} if 7 in detected_ids: # O(1) - list의 in은 O(n) print("장애물 7 추적 중")
```

시간 복잡도 — 왜 dict/set이 빠를까요?

연산	list	dict / set
끝에 추가	$O(1)$	$O(1)$
인덱스 접근 <code>a[i]</code>	$O(1)$	—
포함 검사 <code>x in a</code>	$O(n)$ — 전부 뒤짐	$O(1)$ — 해시로 즉시
중간 삽입/삭제	$O(n)$	$O(1)$

dict/set은 내부적으로 **해시 테이블**을 씁니다. 키를 해시 함수에 넣어 "몇 번째 서랍에 있는지"를 바로 계산하기 때문에 크기와 무관하게 빠릅니다. 그 차이가 실제로 얼마나 벌어지는지 봅시다.



데이터가 커질수록 벌어진다 — 루프 안이라면 곧바로 마감(deadline) 위반으로 이어진다

images/05_시간복잡도.png — 포함 검사 비용

💡 **해석:** 원소가 100개일 때는 둘 다 순식간이라 차이를 못 느낍니다. 하지만 데이터가 쌓여 100만 개가 되면 list는 100만 번을 뒤지고 dict/set은 여전히 한 번에 찾습니다 — **약 2,500배** 차이입니다. 30Hz로 도는 인지 루프 안에서 `in` 검사를 list로 하면, 처음엔 잘 돌다가 시간이 지날수록 마감을 놓치는 **성능 버그**가 됩니다. 2강에서 배운 주기·지터 문제와 직결됩니다.

💡 **팁:** 노트북에서는 `%timeit x in my_list` 로 이 차이를 그 자리에서 직접 재볼 수 있습니다.



선택 기준 한 줄 요약: 순서·중복 필요 → list / 불변 묶음 → tuple / 이름으로 조회 → dict / 수순·중복제거 → set